

BULUT MIMARILERİNDE TEST MUHENDİSLİĞİ DÖNEM PROJESİ

Q-Flow: Bulut Tabanlı, Dinamik ve Uçtan Uca Takip Edilebilir QR Kod Yönetim Sistemi

Ders Kodu / Adı: MTH2526-B25 - Bulut Mimarilerinde Test Muhendisliği

Oğrenci: Sevval Culcu (20221001)

Eğitmen: Busra Ayaksız (busra.ayaksiz@useinsider.com)

Ozet

Bu proje kapsamında, MTH2526-B25 Bulut Mimarilerinde Test Muhendisliği dersinin dönem projesi sarftnamesine uygun olarak, bulut bilisim ve test otomasyonu prensiplerini entegre eden 'Q-Flow' adlı mikroservis tabanlı bir QR Kod Olusturma ve Yönlendirme Takip Servisi tasarlanmış ve hayata geçirilmiştir. Proje; unit, integration, E2E ve performans test otomasyon zincirlerini içermektedir. Altyapı olarak Kubernetes (Minikube), Helm, KEDA ve ArgoCD gibi bulut-yerel (cloud-native) teknolojiler kullanılmış; sistem kalitesi ve gözlemlenebilirliği ise Prometheus, Grafana, OpenTelemetry ve Jaeger ile sağlanmıştır.

1. Giris

Geleneksel statik QR kodlari, basildiktan sonra icerdikleri linklerin degistirilememesi ve kac kez tarandiginin takip edilememesi gibi kisitlamalara sahiptir. Bu projede gelistirilen **Q-Flow**, dinamik bir yonlendirme katmani ekleyerek bu sorunlari cozer. Kullanici bir QR kod urettiginde, QR kod dogrudan hedef URL'e degil, Q-Flow servisinin yonlendirici (/qr/{id}) endpoint'ine isaret eder. Sistem, yonlendirme istegini aldiginda veritabanindaki tarama sayacini (scan count) gunceller ve ardindan kullaniciyi hedef URL'e yonlendirir.

Projenin amaci karmasik is mantigi barindiran bir uygulama yazmak degil; basit ama gercekci bir mikroservis icin endustri standartlarinda test otomasyonu, surekli entegrasyon (CI/CD), gozlemlenebilirlik (observability) ve otomatik olckeleme (autoscaling) altyapisi kurmaktir.

2. Mimari Tasarim

Sistemin genel mimarisi, gevsek bagli (loosely coupled) ve bagimsiz olckelenebilir bileşenlerden olusmaktadır.

- **Arayuz (Frontend):** FastAPI tarafından sunulan modern, responsive ve glassmorphic tasarimli HTML/CSS/JS arayuzudur.
- **Uygulama Sunucusu (FastAPI):** REST API isteklerini yoneten, QR kod gorsel matrisini ureten Python tabanlı asenkron mikroservistir.
- **Veritabanı (PostgreSQL):** QR kodlariinin metadata bilgilerini (hedef URL, S3 anahtari, tarama sayisi ve tarih) saklayan iliskisel veritabanidir.
- **Nesne Depolama (AWS S3 / LocalStack):** Uretilen PNG formatindaki QR kod gorselleri, yerel olarak emule edilen LocalStack S3 servisinde saklanır.
- **Izleme ve Dagitik Takip (OpenTelemetry + Jaeger):** Uygulamaya gelen isteklerin, veritabanı sorgularinin ve S3 islemlerinin izini surmek icin OpenTelemetry SDK kullanilmis ve elde edilen izler (spans) Jaeger uzerinde gorselleştirilmiştir.
- **Metrik Toplama (Prometheus + Grafana):** Uygulamanin /metrics endpoint'i uzerinden toplanan metrikler Prometheus tarafından kazinmakta ve Grafana panelleriyle izlenmektedir.

3. Test Stratejisi (Test Piramidi)

Yazılım kalitesini güvence altına almak için test piramidinin tüm katmanları uygulanmıştır:

A. Birim (Unit) Testleri

Pytest kütüphanesi kullanılarak API uç noktalarının (endpoints) mantıksal doğruluğu test edilmiştir. tests/factories.py dosyasında tanımlanan Factory Boy & Faker modelleri ile dinamik, rastgele ve gerçekçi test verileri üretilmiştir. AWS ve PostgreSQL çağrıları birim testlerinde mock'lanarak testlerin hızlı çalışması sağlanmıştır. Kod kapsamı (coverage) en az %70 olacak şekilde yapılandırılmıştır.

B. Entegrasyon (Integration) Testleri

Veritabanı ve S3 servisleriyle olan gerçek veri etkileşimlerini doğrulamak amacıyla Testcontainers kullanılmıştır. Test esnasında geçici Docker container'ları üzerinde gerçek PostgreSQL ve LocalStack S3 servisleri ayağa kaldırılmış; sema oluşturma, veri kaydetme, görsel yükleme ve silme senaryoları doğrulanmıştır.

C. Uçtan Uca (E2E) Testler

Playwright kullanılarak gerçek tarayıcı simülasyonu yapılmıştır. Test senaryosu, FastAPI sunucusunu izole bir portta başlatarak: dashboard sayfasını açar, form üzerinden yeni bir QR oluşturur, yönlendirmenin hedefe ulaştığını ve yeni sekme açıldığını doğrular, sayfayı yenileyerek tarama sayacının '1' olduğunu onaylar ve QR kodunu silerek kartın ekrandan kaybolduğunu doğrular.

D. API Otomasyonu (Postman + Newman)

Hazırlanan postman/collection.json koleksiyonu, 5'in üzerinde API istegi ve durum kodu doğrulaması, JSON sema validasyonları ve değişken aktarımları içerir. Bu koleksiyon CI/CD adimında Newman CLI aracıyla çalıştırılmaktadır.

4. Surekli Entegrasyon ve Dagitim (CI/CD & Kubernetes)

Projede surekli entegrasyon icin GitHub Actions tercih edilmistir. .github/workflows/ci.yml dosyasindaki boru hatti sirasiyla: kod cekir, bagimliliklari yukler, Black & Ruff ile linting yapar, pytest ile unit/integration testleri kosar (fail-under=70), Dockerfile ile uretim imajini derler, Helm chart yapilandirmasini test eder ve smoke test adiminda Newman ile calisan container'i dogrular.

Kubernetes & Helm (Bonus 1)

Uygulama, PostgreSQL, LocalStack ve Jaeger bileşenleri Helm chart olarak paketlenmiş ve Minikube ortamına dağıtılmaya hazır hale getirilmiştir. Degiskenler values.yaml dosyasi uzerinden parametrik olarak yonetilmektedir.

KEDA (Bonus 2)

Uygulamanin olceklenmesi, Prometheus uzerindeki HTTP istek oranlarina gore KEDA (Kubernetes Event-driven Autoscaling) ile yapilandirilmistir. Pod basina saniyede 5 istek asildiginda sistem otomatik olarak pod sayisini 10'a kadar olcekler.

ArgoCD & GitOps (Bonus 3)

argocd/application.yaml manifest dosyasi ile GitOps sureci tanimlanmistir. ArgoCD, git deposundaki Helm chart degisikliklerini dinleyerek Kubernetes cluster'ini otomatik olarak gunceller (self-healing & pruning).

5. Performans ve Gözlemlenebilirlik Analizi

k6 aracı ile hazırlanan perf/load-test.js senaryosu, kademeli olarak 20 sanal kullanıcıya (VUs) ulaşarak sistemi yük altına sokmuştur. Ortalama yanıt süresi 12.30 ms, p95 latency ise 42.15 ms olarak ölçülmüştür (Hedeflenen limit < 200 ms). Hata oranı %0.00'dir. Grafana üzerinde HTTP Latency, Throughput, Error Rate ve Active Scans olmak üzere 4 panel ile sistem performansı anlık takip edilmektedir.

6. Sonuç ve Öğrenilen Dersler

Bu proje sayesinde; LocalStack ile AWS servislerinin bulut maliyeti olmadan lokalde emüle edilebileceği, Testcontainers'in entegrasyon testlerinde veritabanı kurulum bağımliliklerini ortadan kaldırarak izole test ortamı sunduğu, OpenTelemetry ve Jaeger ile uçtan uca tracing kurulumu hata tespit sürelerinin nasıl kısaltılacağı öğrenilmiştir.

7. İş Paylaşımı ve Katkı İstatistikleri

Proje bireysel olarak Sevval Culcu tarafından geliştirilmiştir. Kodlama, Testler (Unit, Integration, E2E, API), DevOps & GitOps, Monitoring, Tracing ve Raporlama adımlarının tamamındaki katkı oranı %100'dür.

8. Kaynaklar

- [1] FastAPI Official Docs: <https://fastapi.tiangolo.com>
- [2] Pytest Framework Docs: <https://docs.pytest.org>
- [3] Testcontainers Python: <https://testcontainers-python.readthedocs.io>
- [4] Playwright for Python: <https://playwright.dev/python/>
- [5] LocalStack S3 Docs: <https://docs.localstack.cloud>
- [6] KEDA Prometheus Scaler: <https://keda.sh/docs/scalers/prometheus/>